

BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA
ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

ADVANCED INJURY SIMULATION OF GAME CHARACTERS

POKROČILÁ SIMULACE ZRANĚNÍ HERNÍCH POSTAV

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **ADAM HELEŠIC**
AUTOR PRÁCE

SUPERVISOR **Ing. TOMÁŠ CHLUBNA, Ph.D.**
VEDOUCÍ PRÁCE

BRNO 2026

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

Bachelor's Thesis Assignment

172352

Institut:	Department of Computer Graphics and Multimedia (DCGM)
Student:	**Helešic Adam**
Programme:	Information Technology
Title:	**Advanced Injury Simulation of Game Characters**
Category:	Computer Graphics
Academic year:	2025/26

Assignment:

1. Familiarize yourself with game development in the Godot engine.
2. Study the issue of game character animation and damage during combat.
3. Research existing and possible solutions to the given problem.
4. Design a user interface and the final functionality of an application that demonstrates advanced character reactions to injuries.
5. Implement the application and demonstrate its use with different types of data.
6. Perform user or performance testing of the results.
7. Document and publish the final application.
8. Create a video representing the results of your work.

Literature:

- Adams, Ernest, and Joris Dormans. Game mechanics: advanced game design. New Riders, 2012. ISBN 0321820274, 9780321820273.
- Geijtenbeek, T., Vasilescu, D., Egges, A. Injury Assessment for Physics-Based Characters. In: Allbeck, J.M., Faloutsos, P. (eds) Motion in Games. MIG 2011. Lecture Notes in Computer Science, vol 7060. Springer, Berlin, Heidelberg. 2011. DOI: 10.1007/978-3-642-25090-3_7.
- Francis Laclé, Nicolas Pronost. Real-Time Musculoskeletal Model for Injury Simulation on 3D Human Characters. Motion In Games Conference (MIG), Nov 2012, Rennes, France.
- Aristidou, A., Lasenby, J., Chrysanthou, Y. and Shamir, A. Inverse Kinematics Techniques in Computer Graphics: A Survey. Computer Graphics Forum, 37: 35-58. 2018. DOI: 10.1111/cgf.13310.

Requirements for the semestral defence:
Items 1-4, experiments leading to item 5.

Detailed formal requirements can be found at https://www.fit.vut.cz/study/theses/

Supervisor:	**Chlubna Tomáš, Ing., Ph.D.**
Head of Department:	Černocký Jan, prof. Dr. Ing.
Beginning of work:	1.11.2025
Submission deadline:	13.5.2026
Approval date:	7.11.2025

Abstract

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v anglickém jazyce.

Abstrakt

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v českém (slovenském) jazyce.

Keywords

Sem budou zapsána jednotlivá klíčová slova v anglickém jazyce, oddělená čárkami.

Klíčová slova

Sem budou zapsána jednotlivá klíčová slova v českém (slovenském) jazyce, oddělená čárkami.

Reference

HELEŠIC, Adam. *Advanced Injury Simulation of Game Characters.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Tomáš Chlubna, Ph.D.

Advanced Injury Simulation of Game Characters

Declaration

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně pod vedením pana X... Další informace mi poskytli... Uvedl jsem všechny literární prameny, publikace a další zdroje, ze kterých jsem čerpal. Pro jazykovou revizi technické zprávy jsem použil(a) nástroj ChatGPT. Při tvorbě odevzdaného programového kódu jsem využíval nástroj Github Copilot. [konkretizujte a upravte podle skutečnosti].

.....
Adam Helešic
December 30, 2025

Acknowledgements

V této sekci je možno uvést poděkování vedoucímu práce a těm, kteří poskytli odbornou pomoc (externí zadavatel, konzultant apod.).

Contents

1 Introduction **3**

2 Game Character Injury Systems and Animation **4**
2.1 Game Design 4
2.2 Case Studies 5
2.3 Locomotion 6
2.4 Simulating Harm and Injury 8
2.5 Enemy AI 8

3 Injury System Design **10**

4 Implementation and Demonstration in Various Test Cases/Situations **11**
4.1 Creating an Enemy Model 11
4.2 Inverse Kinematics in Godot 11

5 User/Performance testing of the resulting implemented design **12**

6 Documentation and Publication **13**

Bibliography **14**

List of Figures

2.1 Dynamic foot placement using inverse kinematics (The Witcher 3) 7

Chapter 1

Introduction

@TODO: after finishing the thesis

úvod, asi trochu k design philosophy a cílům

The goal of this thesis is the design and implementation of a dynamic injury system of game characters in the Godot Engine. The term *character* here describes game characters in general, and is not intended to be limited to anthropomorphs.

This demo is not player-centric, but rather aims to implement a standalone injury system and its effects on the creature's behavior and animation. That is not to say player experience and game design is thrown out of the window, only that the player controller is not the focus of this demo.

The Godot Engine was chosen for this project because

Chapter 2

Chapter 2

Game Character Injury Systems and Animation

Combat and injury have been a part of human history even since prehistoric times. Singular fights, epic battles, and strifeful wars riddle our existence in stories – mythological through modern, in our personal lives, or in large-scale historical real life events. It is then no wonder that games have used this theme in a myriad of different ways. From mythical, heroic combat with swords and magic to competetive shooters, countless games had to contend with designing and implementing combat and injury systems.

Each game has its budget and own set of priorities based on their design. Sometimes, developers may decide to prioritize other parts of the game like visuals, story, and voice acting, over the implementation of complex combat systems, opting for greater abstraction. The easiest systems are the ones that model characters as a singular entity with no subparts and a pool of hit points that determine its binary state – alive or dead. However, other designs call for a more complex and varied experience. We will look into those in a later section.

In this chapter, we will delve into the basics of game design and the questions that a designer should answer before creating a system for a game. Next, we will look into the subsystems constituting an advanced injury system of game characters – systems of locomotion (movement), tracking and simulating harm and injury through subsystems, and having enemy AI reflect the injuries in its behavior.

2.1 Game Design

Before designing a game system, we need to familiarize ourselves with the theory, practice, and questions of game design.

Firstly, what is game design? *Game design is the act of deciding what a game should be. Decisions about rules, look and feel, timing, pacing, risk-taking, rewards, punishments, and everything else the player experiences is the responsibility of the game designer* [cite Schell book of lenses].

Rules are an essential feature of games. A game's outcome should, to a certain extent, be unpredictable. Games that are fully predictable (a predetermined sequence of events) or fully unpredictable (pure games of chance) aren't engaging. Players want their skills and their strategic decisions to make more of a difference, or else they become frustrated. Skill should also contribute to victory. The longer the game is, the more true this is.

In addition to chance, choices made by players and complex gameplay created by the rules are other ways to make games unpredictable. Complex systems usually have many interacting parts, often simple rules that can combine into interwoven and. difficult to foresee situations. In this type of game, the ability to read a game's current state and understand its strategic complexities is the most important game-playing skill.

Most games mix these three sources of unpredictability. They include an element of chance, player choices, and complex rules. [cite Advanced Game Mechanics]

Now, we can finally get into the core of this thesis and answer the necessary questions about our game design. The goal is to create a more complex combat experience with less abstractions, where the player can make meaningful choices about where to attack an enemy to cripple its capabilities based on those attacks. For example, a player that prefers melee combat might decide to target an enemy's wings first to make it lose its ability to fly to hinder its maneuverability and disable it from getting out of melee range.

Let's dissect the game design from the perspective of the three sources of unpredictability that should make the game more engaging, as mentioned above. The element of chance comes from the enemy's AI which makes randomized decisions based on weighted transitions in its behavior tree. The player chooses which body parts to attack so that they can disable the enemy's abilities that require those parts to be triggered. The complexity comes from the way body parts serve as requisites for the enemy's capabilities, such as two wings being necessary for flight, at least three limbs for walking, or a stinger tail for poison attacks, and the way they enable and limit what actions it has available in its behavior.

We could also argue that depending on the implementation of the player controller, there are even more elements, such as the player maneuvering around the virtual space to avoid or close in on an enemy, or any of the player's special abilities and attacks with various ranges and cooldowns. However, the player controller is a complex issue of its own. It is not covered by this thesis, but the player's experience is a crucial aspect of games and should be accounted for when discussing of game design ([source book of lenses probably?]).

While assessing, designing, and implementing these features, the goal is also to develop in-engine tools that would make creating such enemies more streamlined in Godot.

2.2 Case Studies

In this section, we will examine the systems within games that implement a body part injury system and their approaches to the design.

In Horizon Zero Dawn and Horizon Forbidden West, components and armor plates can be removed with high tear damage weapons, which deals little to no damage, but reveals weak points or disables attacks. [source https://www.polygon.com/horizon-zero-dawn-guide-walkthrough/21357205/machines-fight-rockbreaker-behemoth-thunderjaw-stormbird-deathbringer-tallneck-corrupted-cauldron/]

In Monster Hunter Wilds, removing body parts is a core part of the experience, gathering them as loot or removing the monsters' dangerous abilities. Different weapon types (slashing, blunt, ranged) deal increased damage to specific enemies' body parts. For example, slashing weapons deal extra damage to tails, while blunt weapons can stun, making the enemy roll on the ground for a period of time. [https://www.polygon.com/monster-hunter-wilds-guide/528353/monster-tails-fangs-claws-horns-how-to-get-sever-cut-break/]

Dead Space has many enemies with weak spots, different for each enemy type. Attacking those parts can limit an enemy's attack or movement. Some enemies can have armor that protects their weak points, forcing the player to focus on another weak point or to

change their position to access weak points hidden on the enemy's back. Enemies can also have weak points that are dangerous to the player, such as explosive sacks damaging the player when destroyed in close proximity, forcing another angle of strategy to the weak point system. The important aspect of this combat system is in how it enforces different strategies that the player needs to apply based on the enemy type's weak point effects. [https://www.dualshockers.com/dead-space-remake-necromorph-weakness/]

Fallout offers a slow-motion targeting system that allows players to target specific body parts. Each body part has its own chance of being hit, modified by distance and cover. The number of attacks a player can make is dependent on the player's action points and the weapon type. Body parts have a status that's separate from the target's overall health. A status bar can be worn down until empty, at which point the limb becomes crippled. Crippled limbs have effects like hindered movement, removing a robot's weaponized arm, or shooting an enemy's head (which in most cases kills them outright). [https://www.videogamer.com/guides/a-beginners-guide-to-combat-in-fallout-4/]

In Xenoblade Chronicles X, the player can target appendages which have their own health pool, separate from the overall one. Destroying appendages disables certain attacks and lowers the enemy's defenses. [https://deltiasgaming.com/xenoblade-chronicles-x-combat-guide/]

Phoenix Point's „free aim" combat allows the player to hit specific body parts based on projectile collisions. Each body part has their independed values (like health) and features. Disabling parts has various effects, such as hindering mobility, disabling the enemy's main weapon, or triggering a bleeding effect. Each part also has a different armor value which reduces damage. [https://www.rockpapershotgun.com/phoenix-point-free-aim-and-part-damage-guide-how-to-use-the-free-aiming-system-for-more-precise-shots]

[illustrative pic?]

2.2.1 Summary

Body parts most often have a feature bound to them – an ability (like a weapon attack), protection (armor that protects weak points or targetable body parts from being targeted), or mobility. Attacking such body parts hinders those features, e.g., slowing an enemy down or disabling an attack. Additionally, body parts may serve as weak points, granting the player an advantage when being hit in a particular way (like stunning enemies by hitting their heads with a blunt weapon in Monster Hunter Wilds).

There are also variable values that modify the limbs, such as a to-hit chance (Fallout) or armor that reduces damage taken (Phoenix Point).

Many games model body parts with hit point pools separate from the overall pool.

In our design, we should also think about aiming for variance that enforces different strategies to be employed by the player (like in Dead Space).

We should also be mindful of not overwhelming the player - diagetic combat system (where you hit == that part takes the damage) rather than 8 UI popups.

2.3 Locomotion

Locomotion is the problem of making game characters move. Often, it isn't a single system, but an intertwining of AI, gameplay, physics, and animation systems. cite [Animation-Driven Locomotion with Locomotion Planning, Jarosław Ciupinski]



Figure 2.1: Dynamic foot placement using inverse kinematics (The Witcher 3)

This thesis concerns itself mainly with the enemy AI (behavior tree and pathfinding) and animation. Inverse kinematics is used to make simulated movement look more natural, allowing for end effectors, such as feet, to be placed dynamically based on things like raycast collisions with the terrain mesh (see figure 2.1). Inverse kinematics can also be used for attaching end effectors to the ground (preventing sliding walk animations), reaching for an object (preventing awkward static animations when interacting with objects), etc.

The injury system has to be dynamic, adapting to whichever injuries are applied to the character's body. Hence procedural animation fits this goal of adaptibility well. Rather than traditional animation with preset model movements, procedural animation can account for and adapt to injuries on the run, and make for a more realistic and immersive interaction with the injured character.

2.3.1 Inverse Kinematics

Inverse kinematics is „rotational and translational motion of points, bodies (objects) and systems of bodies (groups of objects) without consideration of what causes the motion or any reference to mass, force or torque“

It had originally been used in robotics for moving arms to targets, but has also expanded into animation in computer graphics. In the latter field, it is very useful for animating *articulated figures* (figures with a specific, solid form like humans, animals, and generally creatures with limbs, as opposed to blobs of jelly, for example). These models are often complicated, and require efficient calculation of complex and multi-link models that can have specified constraints (e.g., a maximum angle that a link can turn by).

[description of IK basic mechanisms and illustrative images - end effectors, end points and target points, joints and links]

The multi-link models in inverse kinematics are usually calculated by points (target positions to reach and the model's joints, known as *end effectors*) and their potential constraints. These end effectors are specified by the model (the joints, the links between them, and the constraints limiting their movement) or by the animation system (the target points that a multi-link model, such as an arm, needs to reach to accomplish its task).

A model is a series of links connected by joints. Joints are the main component allowing movement – their positions determine the positions and orientations of the links. Links are the „connective tissue“ between joints and are defined by their length, shape, volume, and mass properties. A *posture* is the skeletal configuration of the figure and must most often satisfy constraints to adhere to realistic movement, such as the torque limitations of each joint or the links not intersecting with other models in their path. A skeletal configuration is usually separated into *chains*, standalone parts attached to a static point on the model (e.g., the limbs of animals and anthropomorphs). The chains usually work in a hierarchy, and the movement of one joint affects the movement of the ones before it.

[illustration image of chained movement effects]

Usage of Inverse Kinematics

Forward kinematics (traditional animation where known, predefined transformations are applied to the model) can fall short in situations where we might need to dynamically adjust the target locations, such as characters standing on slopes or an appandage reaching for a target that can be a variable distance or height away. Inverse kinematics is adaptive, the movement of the links (such as limbs) is dynamically calculated and placed, making for natural-looking movement. This quality is what makes it an ideal animation method for our dynamic limb-based injury system.

Inverse Kinematics in Godot

2.4 Simulating Harm and Injury

The implementation of injury depends on how we choose to straddle the line between realism and abstraction for the sake of gameplay. One avenue would be to simulate the game character as a physics-based entity with detailed models of musculature and detection of injury precisely where a collision with a weapon happens, and disabling articulated movement that uses the muscles in those parts, etc. However, the goal of this thesis is not to create a perfectly realistic body simulation, but to provide a framework and streamline creating enemies with targetable body parts that serve as requisites for enemy abilities in Godot.

There are many possible implementations of a body part injury system. HP-based; body-part based; idk

probably going with a joint HP pool where each body part is some fraction of that pool

2.5 Enemy AI

The enemy AI will require the solving of two main problems – deciding and executing behavior, and pathfinding. The behavior also needs to be limited deciding on actions and movement; limiting options based on harm; procedural animation of actions

2.5.1 Behavior

Finite State Machine

Behavior Trees

any other ways to implement AI

Behvaior Execution

what pattern to use when simulating the behavior? something that will change states like „Fly Up“, „Descend“, „Rake with Claws“, but what will take care of the actual behavior? And what is the central brain that manages the state?

Behvaior Limiting

patterns to check for available actions? how to limit effectively what the Behavior Tree (or the chosen way to implement behavior) has access to? so that when the random value in the weighted network is generated, so that it doesn't try to execute a branch that cannot be executed due to missing links

[PATTERN DIAGRAMS]

2.5.2 Pathfinding

necessary theory for implementing pathfinding in Godot

Chapter 3

Injury System Design

Navrhněte uživatelské rozhraní a výslednou funkcionalitu aplikace, která demonstruje pokročilé reakce herní postavy na zranění.

Design justified by design principles (books).

Godot plugin design and mockups

Chapter 4

Implementation and Demonstration in Various Test Cases/Situations

4.1 Creating an Enemy Model

Blender. Explain enemy design (body parts) – that's why it's a custom model. sculpting, retopo, rigify addon skeleton edited to fit the model, automatic weight paints with some manual adjustments, exported to Godot.

4.2 Inverse Kinematics in Godot

No need to reinvent the wheel, just make sure it works well. Using Godot 4's SkeletonIK3D and LookAtModifier3D.

adjusting based on centroid (e.g., different foot placement and leaning if a leg is gone)

Chapter 5

User/Performance testing of the resulting implemented design

Chapter 6

Documentation and Publication

+ video

Bibliography